

Advanced and Distributed Algorithms Storing Graphs in C++

Pr. Benoît PIRANDA, Ph.D.

Member of Computer Science Department of Femto-ST institute

benoit.piranda@femto-st.fr



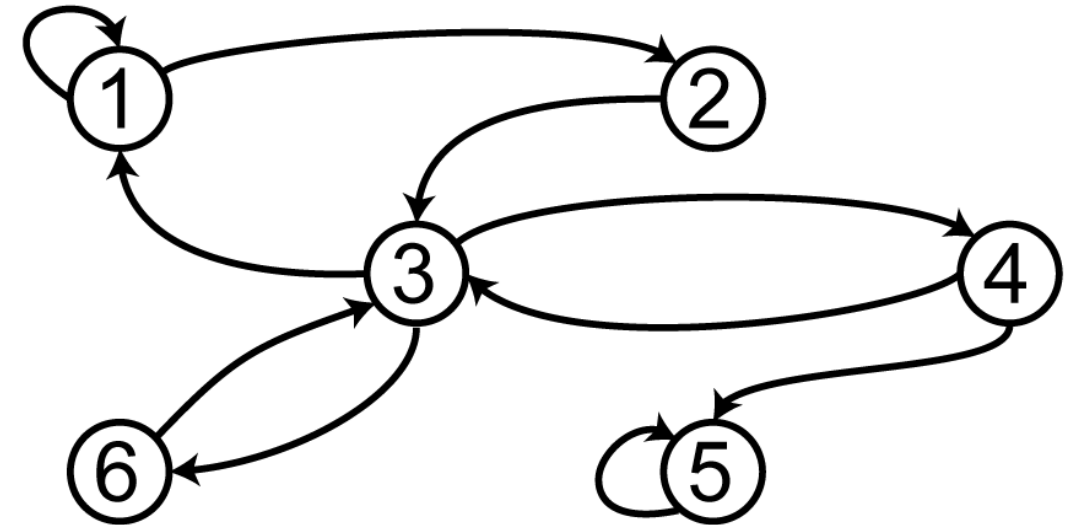
Outline

- Graphs representations
 - Matrix
 - Arrays
- Definition using object
 - Hierarchy
 - Vertices
 - Edges
- Eccentricity and diameter



Generality on Graphs

- Definition:
 - $G(V,E)$
 - V : set of vertices
 - E : set of Edges
 - Connect two vertices
 - Oriented graph: $V_1V_2 \neq V_2V_1$
 - Allow to describe state problems



$$V = \{1,2,3,4,5,6\}$$

$$E = \{(1,1); (1,2); (2,3); (3,1); (3,4); (3,6); (4,3); (4,5); (5,5); (6,3); \}$$

$$|V| = 6 \quad |E| = 10$$

Example of state problem

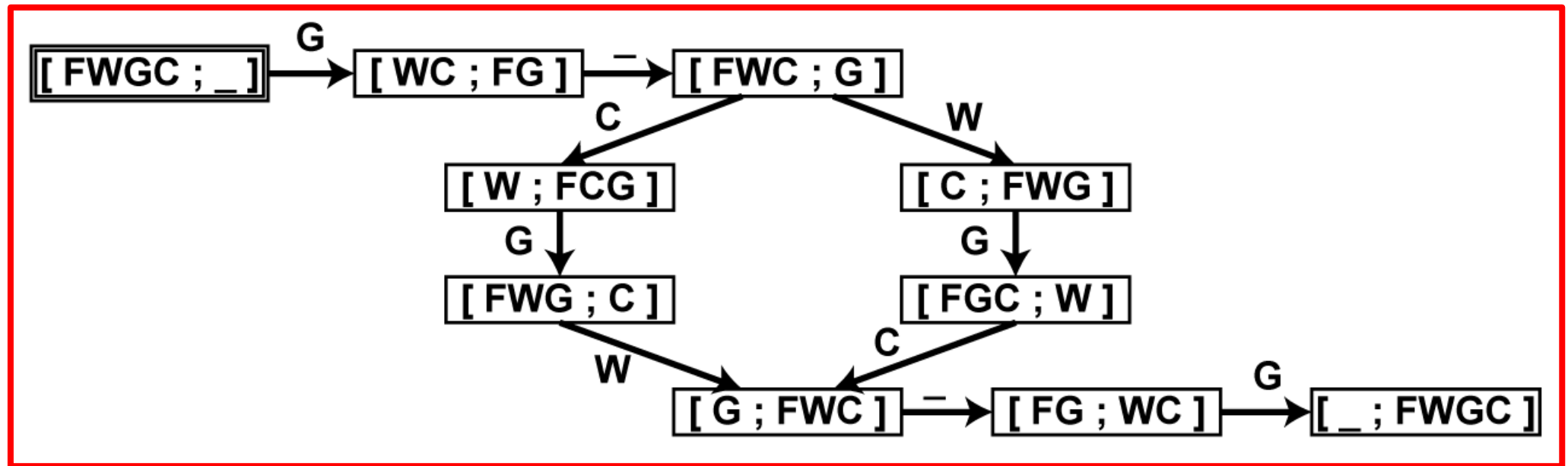
- The Puzzle of the Farmer, Wolf, Goat and Cabbage
 - A Farmer (F), a Wolf (W), a Goat (G) and a Cabbage (C) have to cross a river using a boat that can only accommodate the farmer and at most one other entries at a time.
 - Wolf and Goat cannot be together without the farmer.
 - Goat and Cabbage cannot be together without the farmer.
 - Draw a graph $G=(V,E)$ where
 - V is a list of every valid states $[A;B]$ where A is the list of items that are in the initial bank of the river and B is the list of items that are in the destination bank
 - for example $[FWGC;_]$ and $[WC;FG]$ are valid states, we use $_$ symbol to represent an empty bank.
 - E is a list of triplets $\{I,X,F\}$ precising that the valid state F is obtained from valid state I if the farmer cross the river with item X
 - for example $\{[FWGC;_],G,[WC;FG]\}$ is in E

The Puzzle of the Farmer, Wolf, Goat and Cabbage

- What are initial and final states of the problem?
- Which different values X can take in links triplets?
- Draw the graph of all valid states?

[FWGC;_] [;FWGC]

$X \in \{W, G, C, \emptyset\}$

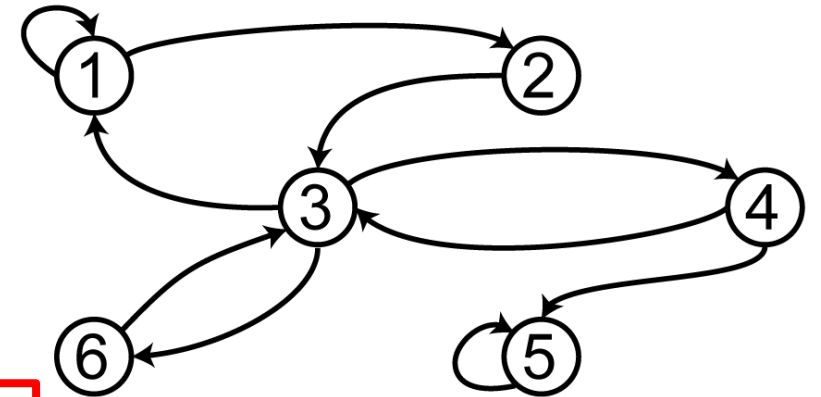


Adjacency lists

- Definition

- List of neighbors of each vertex.

- $\{(V_1: \{E_{1,1}, E_{1,2}, \dots, E_{1,n_1}\}), (V_2: \{E_{2,1}, E_{2,2}, \dots, E_{2,n_2}\}), \dots\}$

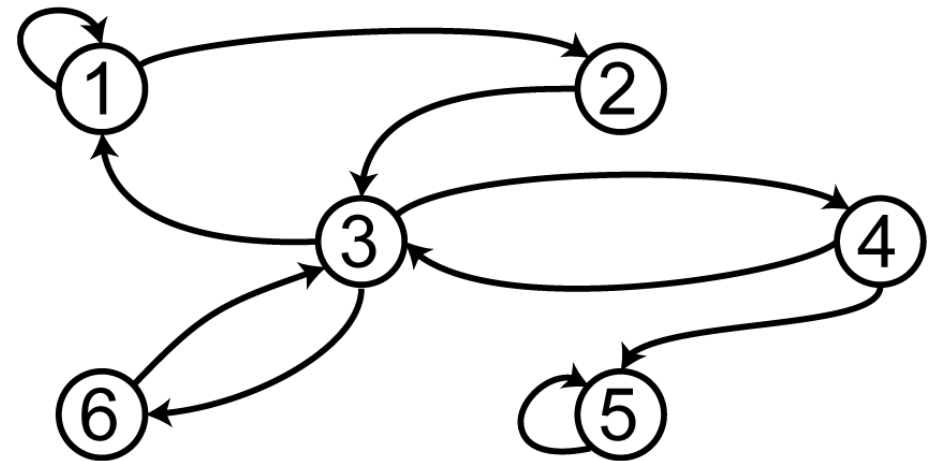


$\{(1: \{1,2\}), (2: \{3\}), (3: \{1,4,6\}), (4: \{3,5\}), (5: \{5\}), (6: \{3\})\}$

Adjacency matrices

- Definition
 - Square matrix which elements indicate whether pairs of vertices are adjacent in the graph.

$$\begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$



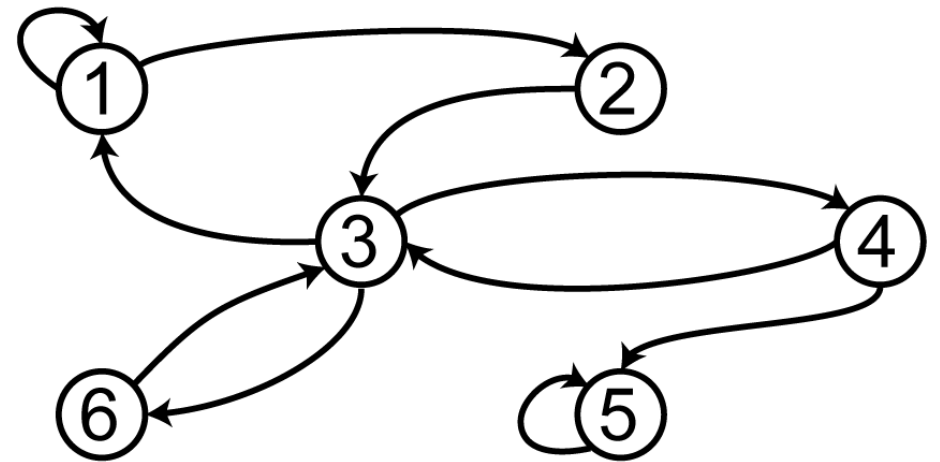
Incidence matrix

- Definition

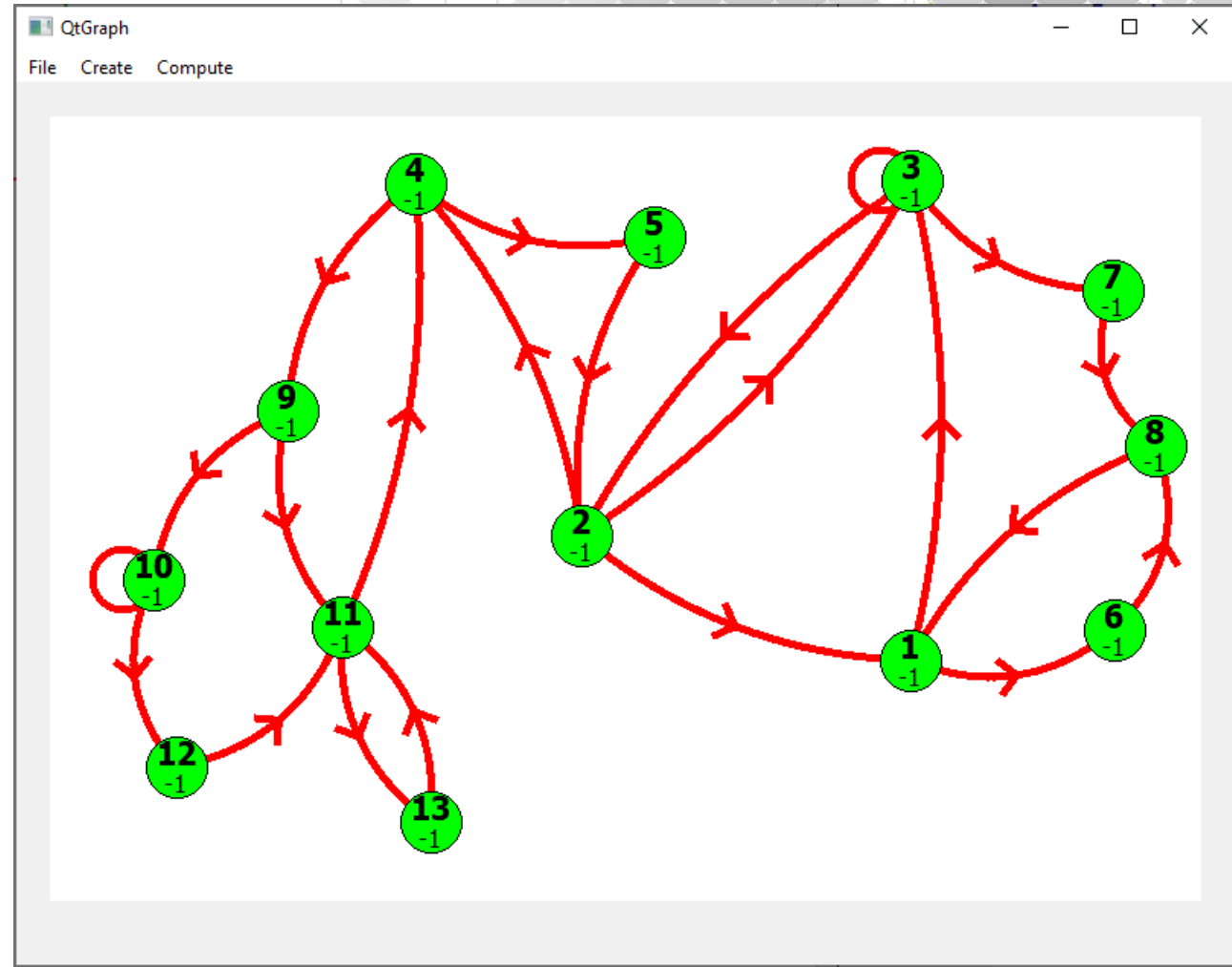
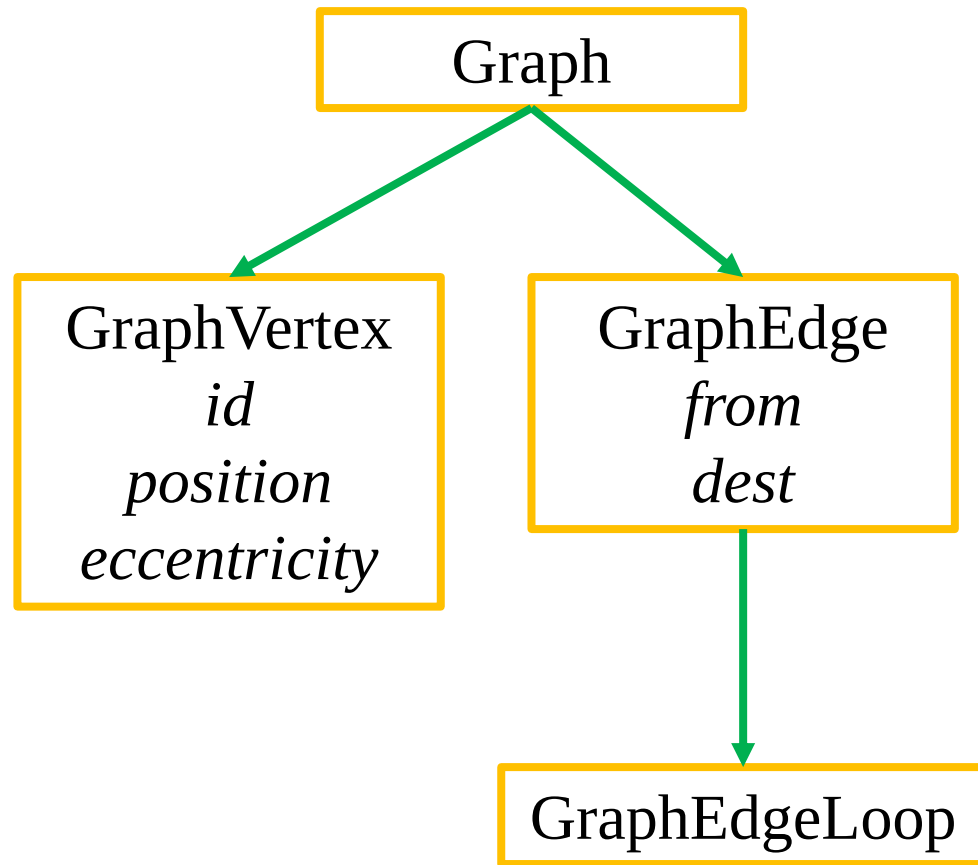
- The incidence matrix of a graph $G(V,E)$ is a $n \times m$ matrix B , where $n = |V|$ and $m = |E|$, such as $B = (b_{ij})$ is defined by:

$$b_{ij} = \begin{cases} -1 & \text{if the edge } j \text{ comes from } i \\ 1 & \text{if the edge } j \text{ reaches } i \\ 0 & \text{otherwise} \end{cases}$$

	a	b	c	d	e	f	g	h	i	j
1	1	-1	0	1	0	0	0	0	0	0
2	0	1	-1	0	0	0	0	0	0	0
3	0	0	1	-1	-1	-1	1	0	0	-1
4	0	0	0	0	1	0	-1	-1	0	0
5	0	0	0	0	0	0	0	1	1	0
6	0	0	0	0	0	1	0	0	0	1

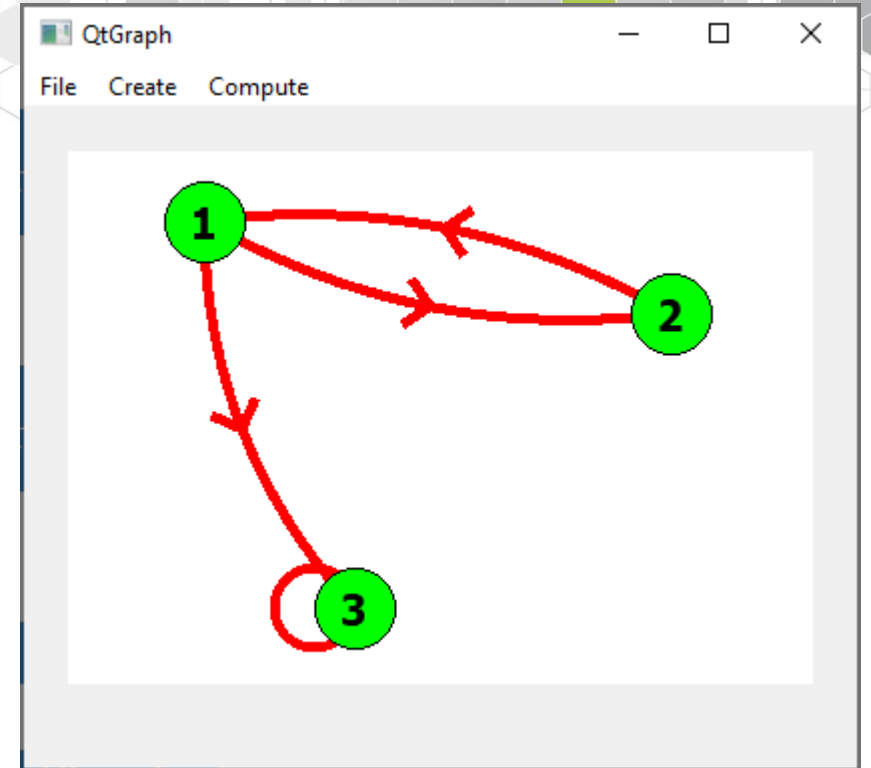


Classes for graphs



JSON format data

```
{  
  "vertices": [  
    { "id": 1, "position": "68,35" },  
    { "id": 2, "position": "301,81" },  
    { "id": 3, "position": "143,228" }  
  ],  
  "edges": [  
    { "dest": 3, "from": 1 },  
    { "dest": 1, "from": 2 },  
    { "dest": 3, "from": 3 },  
    { "dest": 2, "from": 1 }  
  ]  
}
```

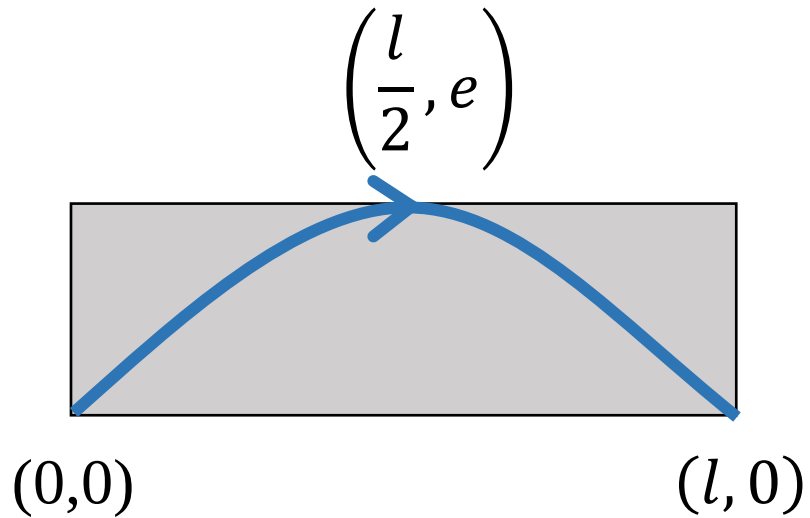


```

void Canvas::loadJSON(const QString &title) {
    QFile file(title);
    if (file.open(QIODevice::ReadOnly|QIODevice::Text)) {
        clearG();
        QString JSON=file.readAll();
        file.close();
        QJsonDocument doc = QJsonDocument::fromJson(JSON.toUtf8());
        QJsonArray JSONvertices = doc["vertices"].toArray();
        for (auto &&v:JSONvertices) {
            QJsonObject vector=v.toObject();
            auto strPosition = vector["position"].toString().split(',');
            GraphVertex *newVertex=new GraphVertex(vector["id"].toInt(),QPoint(strPosition[0].toInt(), strPosition[1].toInt()));
            V.push_back(newVertex); G.push_back(newVertex);
        }
        QJsonArray JSONedges = doc["edges"].toArray();
        for (auto &&v:JSONedges) {
            QJsonObject vector=v.toObject();
            unsigned int val = vector["from"].toInt();
            auto from = std::find_if(V.begin(),V.end(),[val](const GraphVertex *gv) { return gv->getId()==val; });
            val = vector["dest"].toInt();
            auto dest = std::find_if(V.begin(),V.end(),[val](const GraphVertex *gv) { return gv->getId()==val; });
            GraphEdge *newEdge;
            if (from==dest) { newEdge=new GraphEdgeLoop(*from,*dest); } else { newEdge=new GraphEdge(*from,*dest); }
            E.push_back(newEdge);
            G.push_back(newEdge);
        }
    }
}

```

Some drawing tools



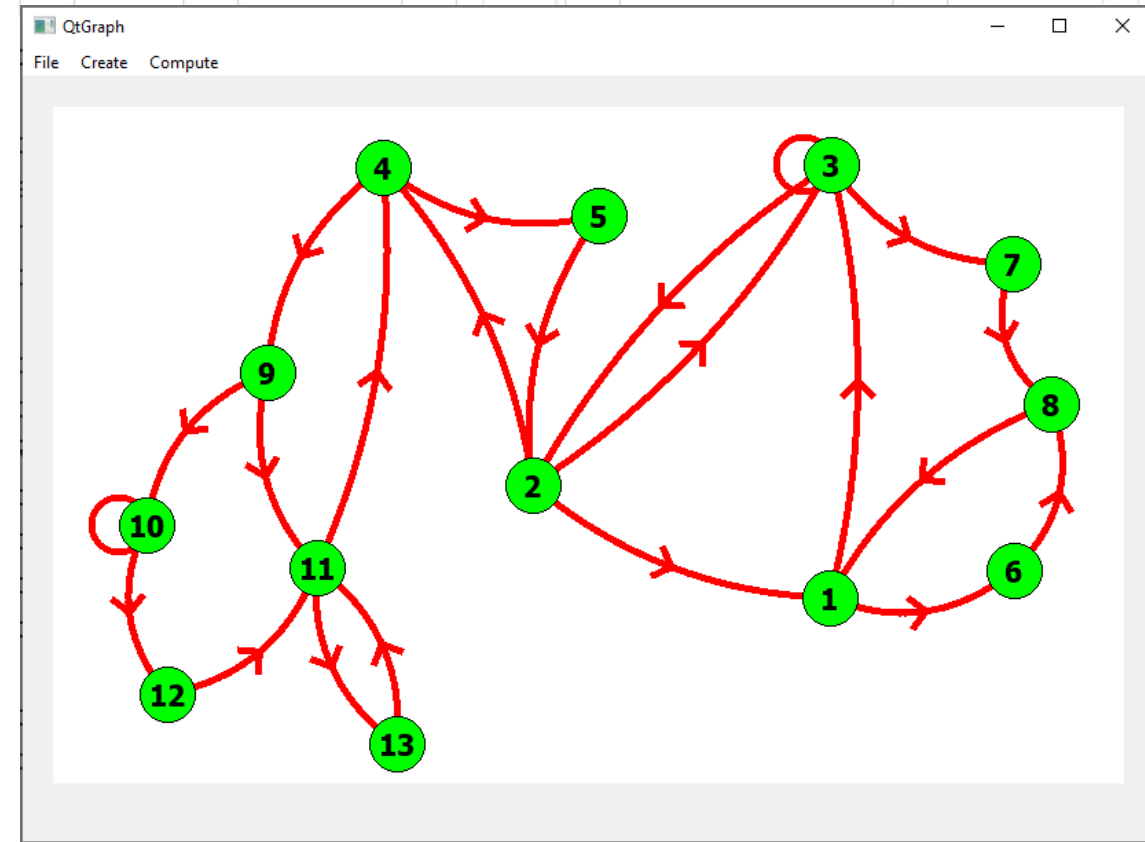
$$f(x) = ax^2 + bx + c$$

$$f'(x) = 2ax + b$$

$$\left\{ \begin{array}{l} f(0) = 0 \rightarrow c = 0 \\ f(l) = 0 \rightarrow al + b = 0 \\ f\left(\frac{l}{2}\right) = e \rightarrow \frac{al^2}{4} + \frac{bl}{2} = e \\ a = \frac{-4e}{l^2} \\ b = -al \end{array} \right.$$

Drawing arrows

```
painter->save();
painter->translate(from->getPosition());
painter->rotate(angle);
QVector<QLineF> segments;
float x0=0.0f,y0=0.0f,x1,y1;
float a=-4.0*(radius)/(length*length),b=-a*length;
for (int ix=1; ix<20; ix++) {
    x1=ix*length/20.0;
    y1=(a*x1+b)*x1;
    segments.append(QLineF(x0,y0,x1,y1));
    x0=x1; y0=y1;
}
x0=0.5*length; y0=(a*x0+b)*x0;
segments.append(QLineF(x0,y0,x0-radius/2.0,y0-radius/2.0));
segments.append(QLineF(x0,y0,x0-radius/2.0,y0+radius/2.0));
painter->drawLines(segments);
painter->restore();
```



Eccentricity



- Definitions
 - ***Distance:***
 - The distance between two vertices is the **number of edges on the shortest path** connecting them.
 - ***Eccentricity:***
 - The maximum distance of one vertex to all other vertices.
 - ***Diameter:***
 - The maximum eccentricity of any vertex in the graph.

Compute the eccentricity of a vertex v

- Compute the distance from v to all the other nodes
 - *Store vertices distances in a map indexed by the id of the node*
- *Initially all distances are ∞ but $\text{distance}[v] = 0$.*
- *Use a list L to store the updated vertices*

```
while  $L$  is not empty
|  $v_0 = L.pop()$ ;
| for all ( $v_1$  connected to  $v_0$ )
| | if ( $\text{distance}[v_1] > \text{distance}[v_0] + 1$ )
| | |  $\text{distance}[v_1] = \text{distance}[v_0] + 1$ 
| | |  $L.push(v_1)$ 
```

Load specifications

- To Do:
 - List of classes to store $G(V,E)$
 - Draw a graph with Vertex id and Links
 - Load a graph from JSON files
 - Print the size $|V|$, $|E|$ and the total number of link pointers stored in vertices in the status bar.
 - Compute and draw the eccentricity
 - Move a vertex with the mouse
 - Add a new vertex (new id and at the center)
 - Add a new edge
- Option
 - Remove a vertex
 - Remove an edge
 - Save the graph in a JSON file

